Settings

Every setting a screener tool stores: what it does, its default, what values it accepts, which code reads it, and where it appears on the **Settings** tab. Written from `src/routes/settings.js` (`SETTING_RULES`), the seed in `src/db/index.js` and the code that reads each key.

Related: the jobs on the same tab (`docs/MONITOR.md`), the data-source check (`docs/DATACHECK.md`), the routes (`docs/API.md`).

How settings work

- **Per tool.** Each tool keeps its settings in the `settings` table of its own database (`data/tradedesk.db` for T1, `data/t<n>.db` for the others). Changing T2's floor does not change T7's.
- **Seeded once.** A fresh database gets the defaults below with `INSERT OR IGNORE`, so a saved value is never overwritten by a restart.
- **Allowlisted.** `POST /api/settings` takes `{ key: value, … }`. A key not in `SETTING_RULES` is refused, every value is checked against its rule, and if any one fails, nothing is saved.
- **Read when used.** Code reads the table at the moment it needs the value, so a saved change applies from the next scan; no restart is needed.
- **Secrets are masked.** `finnhubApiKey`, `alpacaApiKey`, `alpacaApiSecret` and `aiApiKey` are returned as `set` or empty, never as the value.
- **The floor reports what is in force.** `GET /api/settings` starts from the floor values actually applied (`tradable.thresholds()`), so a fresh tool's fields never render blank while a floor is being applied.

Value types:

Type	Accepts	Stored as
int	a whole number inside `min…max`	the number as text
float	a number inside `min…max`, or empty (clears it)	the number as text
str	any text up to `maxLen` characters, trimmed	the text

Live settings (read by the screener tool)

Hot Sector

When a sector is marked 🔥 HOT (`src/sideD/sectors.js`). The sector score is the best score the sector reached in a scan (`docs/MARKET-TAB.md`).

Key	Default	Accepts	Meaning
`hotImmediateThreshold`	60	int 0–100	a score at or above this makes the sector hot at once
`hotSustainedThreshold`	40	int 0–100	a score at or above this counts one session towards “sustained”
`hotSustainedSessions`	3	int 1–20	this many sustained sessions in a row make it hot
`hotFloorThreshold`	20	int 0–100	once hot, it stays hot while its score stays at or above this
`coolOffDays`	2	int 0–30	sessions below the floor before a hot sector cools (despite the name, it counts sessions)

Reset Hot State (`POST /api/settings/reset-hot`) clears the remembered hot/sustained counters; the next scan rebuilds them under the current values.

Sector Bias Thresholds

Key	Default	Accepts	Meaning
`sectorBullishThreshold`	20	int 0–100	a sector score above this is BULLISH
`sectorBearishThreshold`	−20	int −100–0	a sector score below this is BEARISH

Between the two, both included, is NEUTRAL.

Tradability floor

Added to **every** screener of this tool at scan time (`src/sideA/tradable.js`). Zero switches off that one leg; a blank value means the default, never “off”. A screener marked `liquidity: false` drops the two “could you get out” legs (average volume and ATR in dollars); price and ATR % still apply.

The section title says “every screener, every tool”, but the values are stored in **this tool’s** database: changing them on T2 does not change T7 (D-T5).

Key	Default	Accepts	Meaning
`minPrice`	1	float 0–10000	price at least this many dollars
`minAvgVolume`	1000000	int 0–100000000	average daily volume at least this
`minAtr`	1	float 0–100	ATR (14) at least this many dollars
`minAtrPct`	3	float 0–100	ATR at least this % of the price

`minPrice` is not seeded into the table; its default comes from `tradable.js` until it is saved once. The other three are seeded.

Shortlist Auto-Rule

Run one minute before R1 (`src/sideF/shortlist.js`, `docs/MONITOR.md`).

Key	Default	Accepts	Meaning
`shortlistMinScore`	70	int 0–100	only cards with a score at or above this
`shortlistTopN`	5	int 1–50	at most this many, highest score first

The rule needs a score. With the scorer off, no card has one, so it never adds anything (D-D1).

Scorer

Read by `src/sideE/score.js`.

Key	Default	Accepts	Meaning
`scorerEntryTime`	9:40	str ≤10	which entry the score is for: `9:37` (R4A, bases B1–B3) or `9:40` (R4B, bases B4–B6)
`regimeSampleThreshold`	150	int 1–500	a regime-specific table is used only with at least this many samples; below it, the main table

Keys and accounts

Key	Default	Accepts	Read by
`finnhubApiKey`	empty	str ≤100	`src/sharedKeys.js`, for the Finnhub news source
`alpacaApiKey`	empty	str ≤100	`src/sharedKeys.js`, for Alpaca news and bars
`alpacaApiSecret`	empty	str ≤100	`src/sharedKeys.js`
`alpacaAccountUrl`	`https://paper-api.alpaca.markets`	str ≤200	`src/alpaca/client.js` (which Alpaca account the key belongs to)
`alpacaMarketFeed`	not seeded (`iex` when absent)	str ≤10	`src/alpaca/client.js` (`iex` or `sip`)

Where a key is looked for, first found wins (`src/sharedKeys.js`):

1. this tool's setting;
2. `data/keys.json`, shared by every tool (mode 0600);
3. the environment (`~/trade-desk.env`).

The shared file is re-read every 30 seconds, so a key added there reaches every tool without a restart. The Settings tab has **no inputs** for these keys today; they are set in `data/keys.json` or the environment.

Test (`GET /api/settings/test/:service`, service `finnhub`, `alpaca` or `ai`) makes one live call with the stored key and says whether it worked.

Dead settings (accepted, never read by the screener tool)

These are in `SETTING_RULES`, so a save is accepted, but nothing in the screener tool reads them. They are left over from code that moved or was removed.

Keys	Why they are there
`trading_*` (14 keys, plus 4 `trading_sideA_*` aliases)	position sizing and gates of the old trading module; only the journal's own database seeds keys with these names
`grading_*` (7 keys)	the journal's grading engine, which reads them from **its own** database, not from a tool's
`analysisEntryType`, `analysisDirectionalBias`, `analysisSuccessThreshold`, `analysisTrainingWindow`	seeded (`A`, `Up`, `1.5`, `90`); nothing reads them
`aiApiKey`, `aiModel`	seeded (`aiModel` = `anthropic/claude-haiku-4-5`); the page has code for them but no inputs

Recorded as D-T1 in `docs/KNOWN-ISSUES.md`.

Things the code does that the page does not say

- **Zero cannot be saved for some keys.** The hot-sector and sector-bias keys are read as `value || default`, so a saved `0` is replaced by the default: `coolOffDays 0` acts as 2, `hotFloorThreshold 0` as 20, `sectorBearishThreshold 0` as −20. The shortlist keys use `??` and do accept 0. (D-T2)
- **Regime threshold default.** The help text says "Default: 10". The seeded value is 150; 10 is only the code's fallback when the row is missing. (D-T3)
- **Finnhub.** The API Keys section says Finnhub "used to live here"; the news code still asks Finnhub when a key is found. (D-T4)

The Settings tab, top to bottom

Section	What it holds
This tool	name, pause / resume (`docs/TOOL-LIFECYCLE.md`)
Data sources	**Check every data source** (`docs/DATACHECK.md`)
Hot Sector	the five hot keys, **Reset Hot State**
Tradability floor	the four floor keys
Sector Bias Thresholds	bullish, bearish
Shortlist Auto-Rule	min score, top N
API Keys	no inputs; says which sources need none (TradingView, Yahoo, SEC)
Scorer	entry time, regime sub-table threshold
Backups	`docs/BACKUPS.md`

The job list and the last scan’s report are on the **Monitor** tab (`docs/MONITOR.md`).